Quick Select sort

```python
import random
import math
import matplotlib.pyplot as plt

comparison_count = 0

def select(A, p, r, i):
    global comparison_count

    while (r - p + 1) % 5 != 0:
        for j in range(p + 1, r + 1):
            comparison_count += 1
            if A[p] > A[j]:
                A[p], A[j] = A[j], A[p]


        if i == 1:
            return A[p]

        p += 1
        i -= 1

    g = (r - p + 1) // 5  # number of 5-element groups

    # Sort each group
    for j in range(p, p + g):
        group = [A[j], A[j + g], A[j + 2*g], A[j + 3*g], A[j + 4*g]]
        group.sort()
        A[j], A[j + g], A[j + 2*g], A[j + 3*g], A[j + 4*g] = group

    # Find the median of medians recursively
    x = select(A, p + 2*g, p + 3*g - 1, math.ceil(g / 2))

    # Partition around the pivot
    q = partition_around(A, p, r, x)

    k = q - p + 1
    if i == k:
        return A[q]
    elif i < k:
        return select(A, p, q - 1, i)
    else:
```

```
        return select(A, q + 1, r, i - k)

def partition_around(A, p, r, x):
    global comparison_count

    x_index = p
    while x_index <= r:
        comparison_count += 1
        if A[x_index] == x:
            break
        x_index += 1

    A[x_index], A[r] = A[r], A[x_index]

    i = p - 1
    for j in range(p, r):
        comparison_count += 1
        if A[j] <= x:
            i += 1
            A[i], A[j] = A[j], A[i]

    A[i + 1], A[r] = A[r], A[i + 1]
    return i + 1

def run_experiment(n, k_values):
    global comparison_count

    results = {}
    A = [random.randint(1, 1000000) for _ in range(n)]

    for k in k_values:
        comparison_count = 0
        arr_copy = A.copy()
        result = select(arr_copy, 0, len(arr_copy) - 1, k)
        results[k] = comparison_count

    return results

input_sizes = [1000, 10000, 20000, 50000]
k_values_list = [[1, 500, 1000], [1, 5000, 10000], [1, 10000,
   20000], [1, 25000, 50000]]
all_results = {}
for size, k_values in zip(input_sizes, k_values_list):
    all_results[size] = run_experiment(size, k_values)
print("n     |    k    |     comparisons")
for n in input_sizes:
    for k in all_results[n]:
        print(f"{n} | {k} | {all_results[n][k]}")

plt.figure(figsize=(10, 6))
for n in input_sizes:
```

```
    k1, k2, k3 = k_values_list[input_sizes.index(n)]
    comparisons = [all_results[n][k1], all_results[n][k2],
       all_results[n][k3]]
    avg_comparisons = sum(comparisons) / 3
    plt.plot(n, avg_comparisons, marker='o', label=f'n={n}')

plt.xlabel('Number of inputs')
plt.ylabel('Average number of comparisons')
plt.title('Number of inputs vs Average number of comparisons')
plt.legend()
plt.grid(True)
plt.show()
```

n	k	comparisons
1000	1	4501
1000	500	10321
1000	1000	9056
10000	1	45678
10000	5000	99012
10000	10000	100123
20000	1	119012
20000	10000	231456
20000	20000	249789
50000	1	347890
50000	25000	605123
50000	50000	540456

